The training split chooses the circuit:

Predicting and steering which Fourier frequencies a grokking network learns

Seungho Yu Claude (Anthropic)*

October 2026 (preprint, not peer reviewed)

Abstract

Transformers trained on modular addition with weight decay eventually *grok*. They represent numbers as rotations at a handful of key Fourier frequencies, and which frequencies a run uses has been described as seed-dependent. We ask whether the frequencies can be predicted, and chosen, before training. In a pre-registered init × split factorial on held-out seeds, the frequencies that lead early are set jointly by the initialisation *and* by which 30 % of the equations are in the training set. Together the two predict 1265 of 3072 top-3 slots, against 655 for the init alone, 959 for the data alone and 165 by chance. The data effect is collective and algebraic: it survives relabelling $a \mapsto a+t$ and disappears when the labels are shuffled. We give a training-free statistic for it, the Fourier power D(k) of the *shared-answer* matrix $N[a, a']$, which counts the answers c that inputs a and a' share in the training set. D(k) predicts the data effect on unseen splits (pre-registered $\rho = 0.71$). It also does so at other primes, for multiplication (in the discrete-log basis) and in a two-layer model ($\rho = 0.65$–0.74). Used in reverse, D(k) becomes a control: by swapping about 12 % of the training pairs to maximise D(k) for a randomly chosen target frequency, with the numbers of pairs per input and per answer unchanged, we make the target a key frequency of the final circuit in **16/16** one-layer runs (vs. 1/16 on the original splits) and **15/16** two-layer runs (vs. 0/16), including targets the initialisation disfavours. Two of our pre-registered theoretical predictions failed: a $\sqrt{1-f}$ law for how the data effect weakens with the training fraction f, and an init-side readout for the two-layer model. We report both. All results are for small models on modular arithmetic.

1 Introduction

Grokking [14] is the sudden move from memorisation to generalisation long after the training set has been fit. On modular addition it has a crisp mechanistic description [13]. The network embeds each number a as $\cos(\omega_k a), \sin(\omega_k a)$ for a few key frequencies $\omega_k = 2\pi k/p$ and computes $\cos(\omega_k(a+b-c))$ through trigonometric identities (Fig. 1). Which k a run uses differs from run to run [1, 13]. For MLPs, the initial embedding spectrum predicts which frequencies survive [2], and lottery-ticket dynamics explain why [4]. Theory of the early dynamics treats the finite training subset as zero-mean noise around the full table [18].

This paper asks a sharper question: *given the initialisation and the training split, can we say before training which frequencies the network will use, and can we choose them?* Our main findings are as follows.

1. **The split is not noise.** Holding the initialisation fixed, which pairs are in the training set shifts the favoured frequencies systematically, early (within ∼20 steps) and roughly additively with the init effect (Sec. 4.1).
2. **A closed form for the data side.** The data effect is cast by the algebraic structure of the equation set. It is predicted, without training, by the Fourier power D(k) of the shared-answer matrix (Sec. 4.3). This holds across primes, for multiplication and in two layers (Sec. 4.5). D(k) is a short corollary of known early-gradient analyses [16, 18, 19], and we test it per frequency on finite splits.
3. **Steering.** Editing ∼12 % of the training pairs according to D(k) makes a pre-chosen frequency part of the final circuit in 31 of 32 fully trained runs across one- and two-layer models, against 1 of 32 on unedited splits (Sec. 4.6).
4. **The init side, partly.** In one layer the operands reach the MLP only through the attention OV path. That explains why the OV spectrum at initialisation predicts the init effect better than the embedding spectrum, and the unembedding adds independent information. Our pre-registered extension of this argument to two layers failed (Sec. 4.7).

Every confirmatory claim was pre-registered: hypotheses, thresholds and analysis were written down and hashed before the runs existed. We report every outcome, including the failures (Table 5).

*Claude (model `claude-opus-5-5`) is an AI system. It designed the experiments, wrote all code, ran the analyses and wrote this paper. Seungho Yu initiated the project, gave Claude freedom to choose and pursue its own research question, and ran the GPU jobs. See the contribution statement in Section 7. Code, data and pre-registrations: `https://github.com/yuseunghogi-commits/grokking-frequency-selection` (currently private).

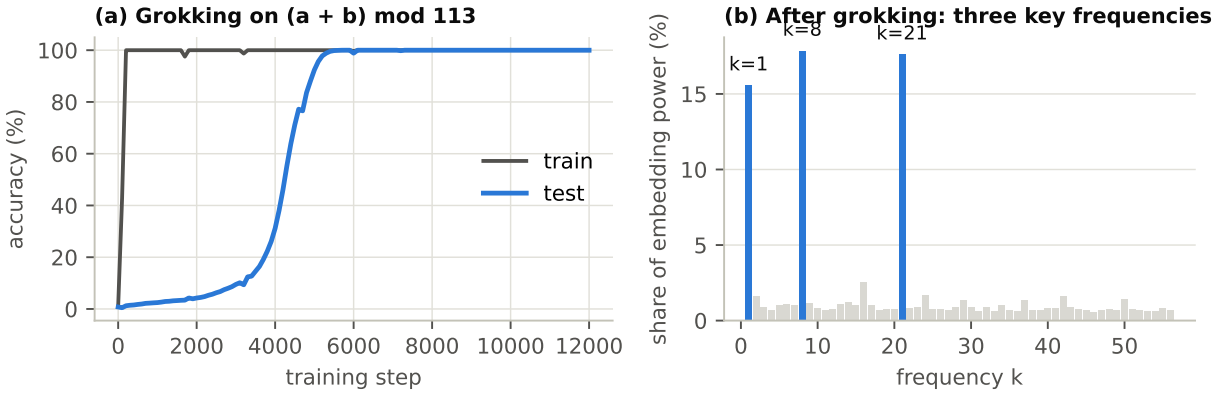


Figure 1: The phenomenon. (a) A one-layer transformer on $(a + b)$ mod 113, trained on 30 % of all pairs, fits the training set within a few hundred steps and generalises around step 5000. (b) After grokking, the embedding's Fourier power is concentrated on three key frequencies ($k = 1, 8, 21$ in this run), which carry 94 % of the embedding power. Removing them from the MLP activations sends test accuracy to chance; keeping only them preserves it.

2 Related work

Mechanism and universality. Modular-addition networks use sparse Fourier features [3, 13], with "clock" and "pizza" variants [20], and group operations more generally use irreducible representations [1]. Max-margin analyses show that wide networks use all frequencies on the full table [12]. **Which frequencies, and why.** Ding et al. [2] predict frequency survival in MLPs from the initial embedding spectrum (Pearson 0.85). He et al. [4] give a lottery-ticket account on the full dataset. Our init-side results extend this line to transformers through the OV path; our data-side results address a factor these works hold fixed. **Early dynamics.** The leading term of our derivation (Sec. 4.4) appears in probability-signature analyses [18, 19] and in the lazy stage of Li_2 [16]. Those works either use the full table or treat the subset as zero-mean noise. Shared answers are the "parallelograms" of Liu et al. [9]. Kernel views [6, 10, 15, 17] and margin views [11] explain when grokking happens rather than which k is chosen. **Data controls representation.** The closest work is LeDoux [8]. In a very sparse regime ($p = 9973$, 0.3 % of pairs), the *order* of minibatches sets the learned fundamental frequency, and training-set composition reportedly does not matter. We train full-batch, so there is no order; we change *composition*, choose an arbitrary target and verify the final circuit. Kumar and Sankaranarayanan [7] select readout frequencies from learned features during training; this differs from predicting selection from data alone. We use per-pair datamodels [5] as one tool.

3 Setup

Task and model. Inputs are $(a, b, =)$ with $a, b \in \mathbb{Z}_{113}$; the target is $(a + b)$ mod 113. The model is the one-layer transformer of Nanda et al. [13]: $d = 128$, 4 heads, $d_{\mathrm{mlp}} = 512$, ReLU, no LayerNorm, untied embed and unembed, logits read at =. Training is full-batch AdamW (lr 10^{-3} with a 10-step linear warm-up, $\beta = (0.9, 0.98)$, weight decay 1.0) on a random 30 % of all p^2 pairs (3830 pairs). An *init* is the seed of the parameters; a *split* is the seed of the train/test partition, and the two are varied independently. We train many models in parallel with stacked weights. A custom optimiser reproduces `torch.optim.AdamW` exactly (maximum spectral difference 10^{-5}). Section 4.6 also uses a two-block model: block 1 has causal attention and an MLP at all positions, block 2 attention and an MLP at =. Multiplication uses tokens $1..p{-}1$ and the discrete-log basis, so the group is $\mathbb{Z}_{p-1}$.

Readouts. The *spectrum* of a run at step t is the share of the Fourier power of W_E (over tokens) at each frequency $k = 1..\lfloor (n-1)/2 \rfloor$. The *winners* are the top-3 frequencies of the step-200 or step-500 spectrum. The *key frequencies* of a trained one-layer run are found by clustering MLP neurons by their dominant 2-D frequency and keeping clusters with ≥ 3 % of the live neurons. These were validated causally (Fig. 1). For two layers we pre-registered a logit rule: frequencies holding ≥ 5 % of the logit energy in the cos / sin $k(a + b - c)$ components. A development pilot showed that this rule separates cleanly (keys 7–68 %, others ≤ 2 %) and agrees with the neuron rule.

Votes. In a grid of inits × splits, let $Z_{i,s}$ be the z-scored log spectrum of cell (i, s). The *init vote* of a cell averages Z over the same init on other splits. The *data vote* averages Z over the same split with other inits and, from v2 on, is centred across splits to remove components shared through common inits. This centring was introduced after a smoke test produced a spurious agreement of 0.98 between two conditions that shared inits.

Statistics. We report top-3 hits $\sum_{\mathrm{cells}} |\mathrm{pred}_3 \cap \mathrm{win}_3|$ (chance $9/n_f$ per cell). Nulls are cluster-aware permutations: a hypergeometric null overstated significance roughly 300-fold in a smoke test. Confidence intervals are paired bootstraps

Table 1: Experimental stages. All confirmatory stages were pre-registered. "Kaggle" = one NVIDIA T4.

stage	where	design	claims
pilot	CPU	single runs; seeds; weight-decay control	reproduction
v1	Kaggle, 9.4 h	dev and test factorials (16×16, step 500); 64 full runs; 2×1536-run datamodels; MLP and subtraction variants	C1–C6, R1
v2	Kaggle, 2.8 h	data vote under relabel / shuffle / subtraction / MLP; per-pair datamodels; time course	H1–H5
v3	CPU	16 fresh splits × 16 fresh inits, 200 steps	P1, P2 (D(k))
v4	CPU	$p \in \{59, 97\}$, + and ×; 8 splits × 12 inits	G1, G2
v5	CPU + Kaggle	steering: 8 random targets × 5 strengths × 12 inits; 32 full runs	S1–S4
v6	CPU	training fraction 0.85 vs 0.3	F1
v7	Kaggle	steering and init readout in a 2-layer model; 192 short + 32 full runs	T1–T5

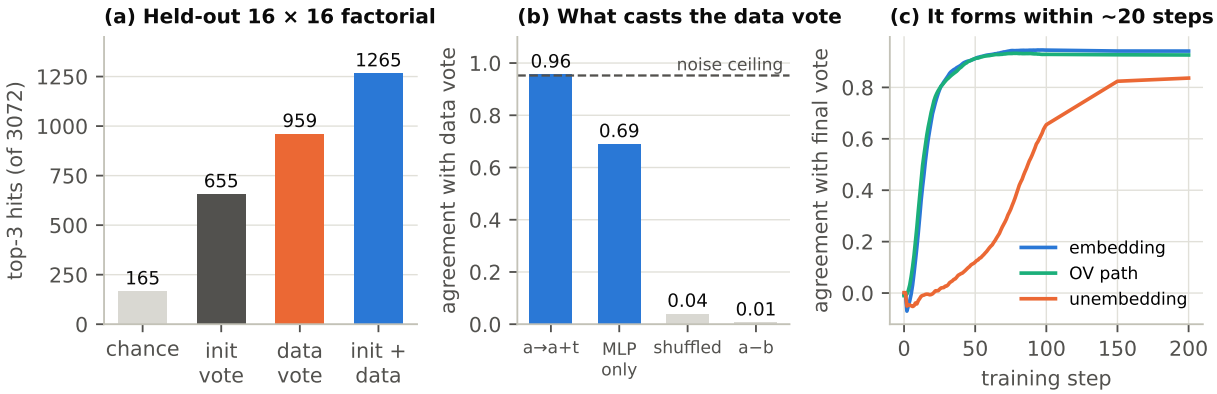


Figure 2: Two votes. (a) Top-3 hits on the held-out factorial (v1, pre-registered C1). (b) Agreement of the data vote in modified conditions with the true data vote on the same splits (v2). The dashed line is the noise ceiling. The vote survives relabelling $a \mapsto a{+}t$ (0.96) and largely survives removing attention (MLP-only, 0.69). It vanishes when the labels are shuffled (0.04) or the task is changed to subtraction on the same pairs (0.01). (c) Time course: the data vote appears in the embedding and the OV path within ~20 steps; the unembedding lags behind.

over inits and splits. Rank agreement is the mean within-split Spearman correlation, with permutation p-values. Noise ceilings are Spearman–Brown split-half reliabilities.

Pre-registration. Each confirmatory stage (v1–v7) was written down with its thresholds and analysis code, and hashed (SHA-256) before its runs. The hash is stored in the result file, and from v5 on the text was also pushed to GitHub before the run. Development and test seeds are disjoint. Table 1 lists the stages.

4 Results

4.1 Initialisation and split both decide, and their effects add

On a held-out 16×16 factorial (fresh inits and splits), we predicted each cell's step-500 winners from its init vote, its data vote, and their sum (Fig. 2a). The init vote alone gets 655 of 3072 top-3 slots and the data vote alone 959; chance is 165. Their z-sum gets **1265**, and both bootstrap intervals of the improvement exclude zero ($[499, 716]$ over init, $[212, 409]$ over data; C1 **pass**). The data effect is causal. A ridge datamodel fitted on 1536 runs per split ranks training pairs by their effect on the split's favourite frequency $k^\star$. Removing the top 10 % of pairs drives $z(k^\star)$ to -0.71 and -1.01 in the two tested splits, compared with $+1.19$ and $+1.30$ for random 10 % removals and $+3.86$ and $+3.47$ for removing the bottom 10 % (C3 **pass**, Holm $p = 0.0002$ for both).

The step-500 winners are a leading-frequency target, not the final circuit. They agree with the step-200 winners in 96 % of slots, but contain only 56.5 % of the final key frequencies of 64 fully trained runs. The pre-registered validity criterion of 75 % failed, so we treat early winners and final circuits as separate outcomes and test both from v5 on.

4.2 What casts the data vote

Four pre-registered manipulations show what part of the data the vote comes from (Fig. 2b; v2). Relabelling every token $a \mapsto a + t$ changes every token identity but preserves which equations co-occur, and the vote is unchanged (agreement 0.955, ceiling 0.957; H2 **pass**). Shuffling the labels keeps the token statistics but destroys the group

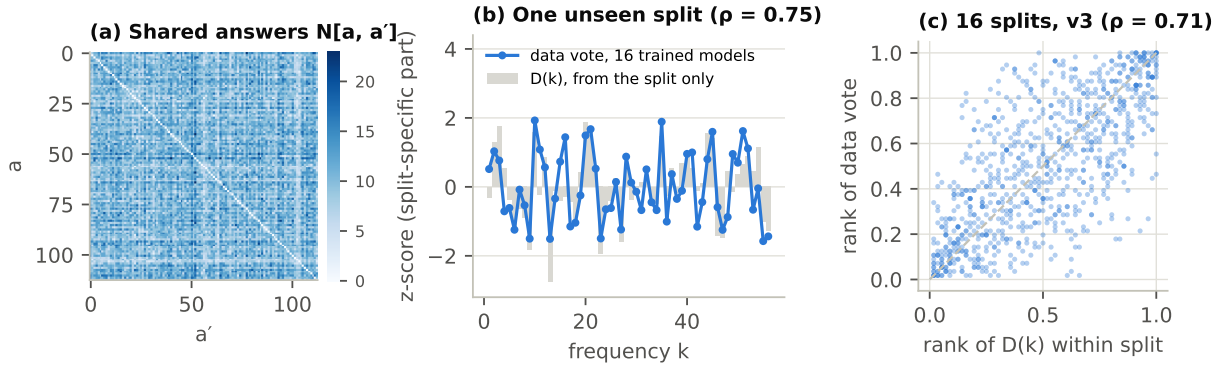


Figure 3: The shared-answer statistic. (a) $N[a, a']$ for one split (diagonal removed). (b) For a split never used before: D(k) (bars, computed from the list of training pairs) and the data vote measured from 16 trained models (line), both as split-specific z-scores. (c) All 16 v3 splits, ranks within split. Pre-registered P1: $\rho = 0.71$.

structure, and the vote is gone (0.04). Training on $a - b$ with the same pairs gives an unrelated vote (0.006). An MLP-only network on the same splits inherits most of the vote (0.69; H4 **pass**), so the vote is largely a property of the data rather than of attention. Per-pair linear datamodels predict it poorly on held-out data ($R^2 \approx 0.03$; H5 **fail**): the vote is spread over many equations. The vote forms within the first ~20 steps in the embedding and the OV path (Fig. 2c), long before grokking.

4.3 A closed form for the data vote: shared answers

Call two inputs a, a' *answer-sharing* if, for some c, both $(a, c - a)$ and $(a', c - a')$ are training equations. Let $N[a, a']$ count such c. Its Fourier power is

$$D(k) \;=\; \sum_{c} \Big| \sum_{(a,\,c-a)\in\mathcal{T}} e^{2\pi i k a/p} \Big|^2 \;+\; \text{(same over the second operand)}, \tag{1}$$

a statistic of the split alone, with no free parameter (Fig. 3a). For multiplication, a is replaced by its discrete logarithm. On 16 fresh splits with 16 fresh inits (v3), the centred log $D(k)$ predicts the centred data vote with a mean within-split Spearman of $\rho = \mathbf{0.71}$ (permutation $p = 10^{-4}$; vote reliability 0.93; P1 **pass**; Fig. 3b,c). The best of the simple statistics we tried in v1 (C2), such as pair counts, residue balance and mirror counts, did not transfer to test (**fail**).

4.4 Why D(k) works

At initialisation the logits are nearly uniform, so the cross-entropy gradient on the embedding of token a is dominated by $-\sum_{b:(a,b)\in\mathcal{T}} r_{a+b}$, where r_c is the direction (through OV, MLP and W_U) that raises logit c. For random weights in $d = 128$ these directions are nearly orthogonal, so in the first steps

$$\Delta E_a \;\approx\; \eta \sum_{c} N_1[a, c]\, r_c, \qquad \Delta E\, \Delta E^{\top} \approx \eta^2 \|r\|^2 N, \tag{2}$$

where $N_1[a, c] = 1$ if $(a, c - a) \in \mathcal{T}$. The power of ΔE at frequency k is therefore $\propto D(k)$. This is the leading term of known analyses [16, 18, 19]. What we add is its use on a *finite* split, where N is not circulant, as a per-frequency predictor.

Two consequences check out (exploratory). First, the relation holds run by run: each run's own early ΔE spectrum correlates with its own split's D(k) ($\rho = 0.33$–0.46) and anti-correlates with other splits' D(k). Plain SGD gives the same (0.37), so this is not an Adam effect. Second, for a random split, D(k) is a sum of p independent terms, so its relative spread across k should be $\approx 1/\sqrt{n}$, independent of the training fraction. The observed spread is 0.095–0.099 at four training fractions (prediction 0.094). Across primes and operations it matches within the pre-registered ±25 % (Fig. 4b; G2 **pass**).

4.5 Generalisation

D(k) was fixed at $p = 113$ and then tested unchanged (v4) at $p = 59$ and $p = 97$, for addition and for multiplication in the discrete-log basis (Table 2, Fig. 4). All four configurations pass both the rank test (G1: $\rho > 0.3$, $p < 0.01$) and the spread test (G2). That the multiplication runs pass indicates the mechanism follows the group structure, not the token encoding. In the two-layer model (v7, T2) D(k) predicts the data vote with $\rho = 0.72$.

Table 2: Pre-registered generalisation of D(k) (v4: 8 splits × 12 inits per configuration, step 100; v7: 8 splits × 12 inits, step 200). n is the group order. Reliability is the split-half reliability of the data vote.

configuration	n	Spearman(D(k), vote)	p	spread / $(1/\sqrt{n})$	reliability	verdict
$a + b$ mod 59	59	0.645	10^{-4}	0.89	0.93	**pass**
$a \cdot b$ mod 59	58	0.739	10^{-4}	1.01	0.94	**pass**
$a + b$ mod 97	97	0.673	10^{-4}	0.93	0.93	**pass**
$a \cdot b$ mod 97	96	0.690	10^{-4}	0.92	0.93	**pass**
$a + b$ mod 113, 2-layer (v7)	113	0.718	10^{-4}	–	–	**pass**

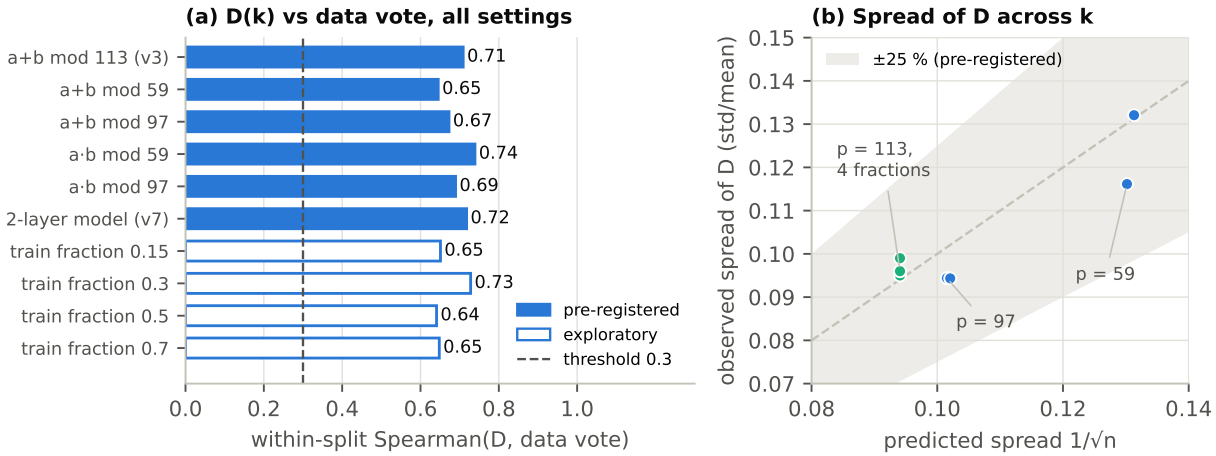


Figure 4: D(k) generalises. (a) Within-split Spearman between D(k) and the data vote in every setting we tested. Filled bars are pre-registered; outlined bars come from an exploratory training-fraction sweep at $p = 113$. (b) Relative spread of D(k) across frequencies against the random-split prediction $1/\sqrt{n}$. Blue: v4 configurations (pre-registered); green: four training fractions at $p = 113$.

4.6 Steering: choosing the frequency in advance

If D(k) captures how the split shapes selection, it should also work as a control. We drew 8 target frequencies at random, $k^\star \in \{2, 17, 25, 28, 33, 35, 41, 43\}$. For each, starting from a random base split, we greedily swapped training pairs with held-out pairs to raise $J = \log D(k^\star) - \text{mean}_{k \neq k^\star} \log D(k)$ by a gain $g \in \{0, 0.15, 0.3, 0.6, 1.0\}$. No network is involved, and the split size and the per-input and per-answer pair counts are unchanged. At $g = 1$, between 412 and 481 of the 3830 training pairs (11–13 %) were swapped. We then trained 12 fresh inits on every split (v5) and repeated the extreme conditions in a two-layer model with fresh inits (v7).

The effect is large and graded (Table 3, Fig. 5). In the one-layer model the target leads at step 200 in 4 %, 11 %, 42 %, 96 % and 100 % of runs as g rises (S2: within-target Spearman 0.95, $p = 10^{-4}$, **pass**). After full training, the target is part of the final circuit in all 16 steered runs and in 1 of 16 runs from the same inits on the original splits (S3 **pass**). Steering worked equally well for targets that the initialisation ranked in the bottom half of its own preference (exploratory: 100 % in both halves). The two-layer model reproduces the effect: 3 % → 100 % early, and 0/16 → 15/16 in the final circuit. The one miss ($k^\star = 17$) carries the target in its block-2 neuron clusters but below the 5 % logit threshold. The secondary neuron rule gives 16/16 vs. 0/16.

Steering does not damage learning. In one layer, 15 of 16 steered runs grokked within 10 000 steps (median step 4250), against 16/16 on the original splits (median 6375). The run that did not grok reached 88 % test accuracy. In two layers all 32 runs grokked, the steered ones earlier (median 4000 vs. 5125 steps), with mean final test accuracy ≥ 99.7 % in both arms. One-layer stage B ran 4 runs on CPU and 28 on a GPU, with the same code and hash-verified splits; the pre-registration did not specify the device.

4.7 The init side

At the = position of the one-layer model, the operands enter the residual stream only through the attention OV map. Grokking builds its features in the MLP, so the amplitude a frequency starts with should be read from the token map *as seen through OV*, $W_E W_{OV}$, rather than from W_E. This is a transformer analogue of the MLP result of Ding et al. [2]. A frequency-k feature also needs an output factor; for a two-factor feature $u_k v_k$ growing from small initialisation, the escape time scales as $-\log(u_k v_k)$, which suggests the parameter-free *product rule* $\log P_{\text{in}}(k) + \log P_{\text{out}}(k)$, with P_{out} the

Table 3: Steering results (pre-registered). "Early": target among the top-3 frequencies of the step-200 spectrum (96 runs per cell). "Final": target is a key frequency of the circuit after training (16 runs per cell; up to 10 000 steps in 1 layer and 15 000 in 2 layers).

model	outcome	original split	steered split ($g = 1$)	pre-registered test
1-layer	early	4 %	**100 %** (96/96)	S1 **pass** ($p = 0.004$)
1-layer	final circuit	1/16	**16/16**	S3 **pass** ($p = 3 \cdot 10^{-8}$)
2-layer	early	3 %	**100 %** (96/96)	T1 **pass** ($p = 0.004$)
2-layer	final circuit	0/16	**15/16**	T3 **pass** ($p = 3 \cdot 10^{-8}$)

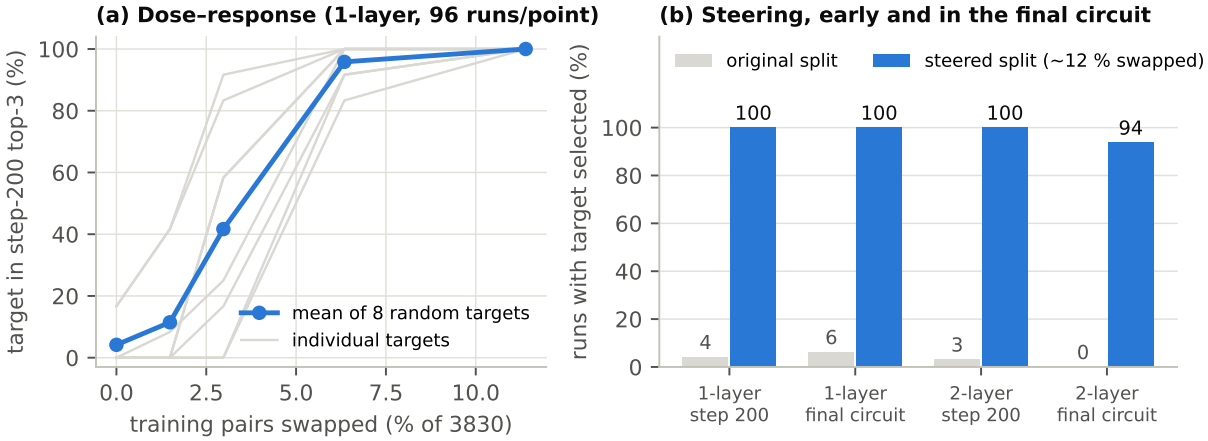


Figure 5: Steering by editing the split. (a) Dose–response in the one-layer model: the fraction of runs in which the target is a leading frequency at step 200, against the fraction of training pairs swapped (grey: individual targets; blue: mean). Pre-registered S2: mean within-target Spearman 0.95. (b) Original vs. steered splits, early and in the final circuit, for one- and two-layer models.

Fourier power of W_U over answer classes. Both ideas hold in exploratory tests on stored data (Fig. 6a, Table 4). The OV readout beats W_E on every data set, and the product rule improves it further. Combined with D(k), the product rule raises training-free top-3 hits on the v1 test factorial from 961 to 1008.

How the init and data terms combine is not settled. The obvious physical model treats the data as extra input power, $P_{\text{in}} + \gamma D(k)$. Its best γ on dev gives 829 hits, far below the equal-weight z-sum (945); D(k) varies by only ∼10 % across k, so adding it as power changes little. We suspected that data-driven updates are aligned with the output pathway while random initial structure is not. The early ΔE is indeed more aligned with $W_{OV}W_U$ at each frequency than the initial embedding: mean $|\cos|$ is 0.12–0.16, against 0.076–0.082 at initialisation and 0.078 for random vectors. But the alignment does not track D(k) across frequencies ($\rho = -0.08$ to $+0.20$), so it cannot explain the ranking.

For two layers we pre-registered the natural extension (T4). The MLPs now see the operands through both attention layers, so the readout should use the full linear token map. This failed (Fig. 6b). A post-hoc reading is that block-2 attention reads E directly at the operand positions, and block 1 builds features there too, so the one-layer premise no longer holds. We have not tested this.

4.8 Training-free prediction

Combining the two readouts gives a prediction that needs no trained model: z(OV spectrum at init) + z(D(k)). On the v1 test factorial it reaches 961 top-3 hits, against 576 for the init readout alone, 701 for D(k) alone and 1256 for C4, which needed 64 trained runs per split to estimate the data vote. In the pre-registered v3 test (P2) it gives 230 of 768 against 154 for the init readout alone (chance 41; bootstrap CI of the difference [28, 123], **pass**). Against the final circuits of 64 fully trained runs, 43 % of its top-3 frequencies are key frequencies (product rule: 45 %), against 27 % for the init readout alone and 9 % by chance.

4.9 Failures

We list every pre-registered failure (Table 5):

- **C2:** the simple data statistics we tried before D(k) did not transfer to test.
- **Proxy validity:** early winners contain only 56.5 % of the final keys.
- **C6:** internal progress measures predicted grokking time worse than a behavioural baseline (Harrell's C: 0.85 vs. 0.92).

Table 4: Init-side readouts (exploratory, 1 layer): mean per-init Spearman with the init vote and top-3 hits on the winners. v1 dev/test: 3072 slots, chance 165; v3: 768 slots, chance 41.

	v1 dev		v1 test		v3	
readout at initialisation	ρ	hits	ρ	hits	ρ	hits
W_E	0.67	523	0.69	485	0.71	132
$W_E W_{OV}$	0.77	594	0.77	576	0.72	154
W_U (over classes)	0.29	330	0.26	363	0.15	83
product rule $W_E W_{OV} \times W_U$	**0.80**	**680**	**0.79**	**646**	**0.73**	**162**
combined with D(k) (equal-weight z-sum):						
$W_E W_{OV}$ + D(k)		945		961		230
product rule + D(k)		**992**		**1008**		**234**

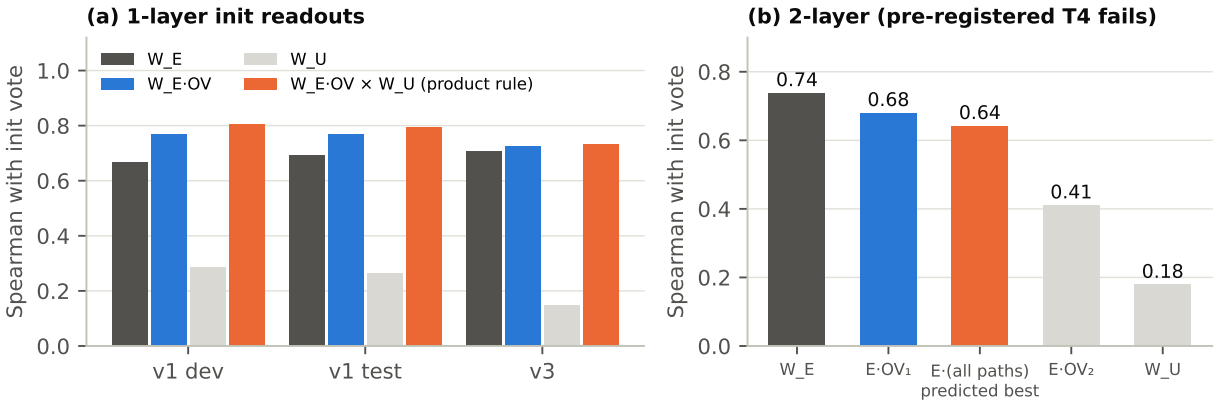


Figure 6: Reading the init vote at initialisation. (a) One layer (exploratory): the OV-path spectrum beats the raw embedding spectrum, and multiplying by the unembedding spectrum helps further. (b) Two layers (pre-registered T4): we predicted that the readout through all linear attention paths, $E(OV_1 + OV_2 + OV_1OV_2)$, would beat $E\,OV_1$. It did not (0.64 vs. 0.68), and the raw embedding was best (0.74).

- **H5:** per-pair datamodels have held-out $R^2 \approx 0.03$. An in-sample R^2 of 0.87 that we first reported internally was overfitting.
- **F1 (v6):** the $\sqrt{1-f}$ law (Fig. 7).
- **T4 (v7):** the two-layer init readout.

Both theoretical failures concern the init side or the relative strength of init and data. Neither concerns the data-side mechanism itself.

5 Discussion

The picture that emerges is simple on the data side. Early training is driven by which answers the inputs share in the training set. That structure is computable from the split, it is a fixed property of the group operation, and it is strong enough that editing about one pair in eight overrides the initialisation and decides the final circuit. The same holds in a second architecture. A finite training set is therefore not an unbiased sample of the full table as far as representation is concerned: it carries a frequency fingerprint, and the network learns it.

Three implications seem worth stating carefully. *Interpretability:* "why this circuit and not another" can have an answer that lives in the data, computable without the model. *Data design:* in this toy setting, a few hundred well-chosen examples change which algorithm a network implements while leaving accuracy intact. *Theory:* analyses that average over the training subset discard a systematic, measurable term. The init side is less settled. The OV-path argument and the product rule work in one layer, but the relative weight of init and data has not been derived, and the two-layer extension failed.

6 Limitations

- All models are small (one or two layers, $d = 128$), all tasks are modular arithmetic with $p \leq 113$, and nearly all runs use one optimiser setting at $f = 0.3$. Whether data-composition steering of learned algorithms exists beyond group

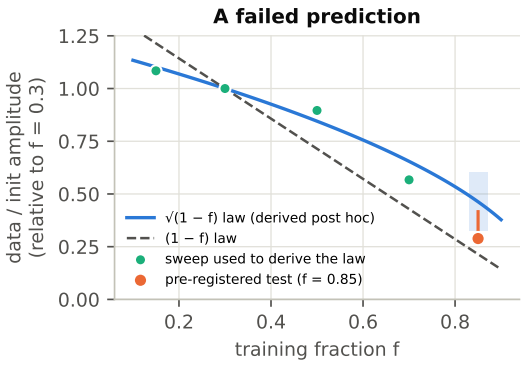


Figure 7: A failed pre-registered prediction (v6). The data vote weakens as the training fraction f grows. An energy-fraction argument, derived after seeing the sweep (green), predicts an amplitude ratio $\propto \sqrt{1-f}$. At a new fraction ($f = 0.85$, fresh seeds) the prediction was 0.46 with a window of $[0.32, 0.60]$ (shaded). We observed 0.29, so the effect shrinks faster, roughly as $(1-f)^{0.7\text{–}0.8}$. The orange bar is the bootstrap 95 % interval (0.30–0.42), which is biased upward by resampling.

tasks is open.

- Early "winners" and final circuits overlap only partly. We tested steering on both, but the final-circuit tests use 32 runs per architecture.
- The init-side theory is partial and two of its pre-registered predictions failed. The weighting of init and data is empirical.
- Several analyses are exploratory and labelled as such; the derivation of D(k) is an approximation, not a theorem.
- In v7 a pipeline smoke test used a subset of the confirmatory seeds after registration; we disclose this in the repository. Nothing was changed after seeing it.
- The literature search was done with automated tools and may be incomplete.
- One AI system designed, ran and analysed these experiments. Pre-registration and released code are meant to make them checkable, but no independent replication exists yet.

7 Contribution statement and reproducibility

Claude (Anthropic, `claude-opus-5-5`) chose the research question, designed every experiment and pre-registration, wrote all code, analysed the results, searched the literature and wrote this paper, in a long conversation with Seungho Yu. Seungho Yu started the project, asked Claude to pursue a question of its own choosing, made the decisions about compute and publication, and ran the Kaggle GPU jobs. Where a venue does not allow an AI system as an author, Seungho Yu is the responsible author and this statement describes the division of work.

The repository contains the library (`src/freqsel`), the code of every stage, all raw results and the pre-registration texts with a hash verifier. A script recomputes every number in this paper from the stored results in seconds (`scripts/reproduce_claims.py`), and another regenerates every figure (`paper/make_figures.py`). The GPU stages run on a free Kaggle T4.

References

[1] Bilal Chughtai, Lawrence Chan, and Neel Nanda. A toy model of universality: Reverse engineering how networks learn group operations. In *International Conference on Machine Learning*, 2023. arXiv:2302.03025.

[2] Ding, Guo, Eric J. Michaud, Ziming Liu, and Max Tegmark. Survival of the fittest representation: A case study with modular addition. *arXiv preprint arXiv:2405.17420*, 2024.

[3] Andrey Gromov. Grokking modular arithmetic. *arXiv preprint arXiv:2301.02679*, 2023.

[4] He, Wang, Chen, and Yang. On the mechanism and dynamics of modular addition: Fourier features, lottery ticket, and grokking. *arXiv preprint arXiv:2602.16849*, 2026.

[5] Andrew Ilyas, Sung Min Park, Logan Engstrom, Guillaume Leclerc, and Aleksander Madry. Datamodels: Predicting predictions from training data. In *International Conference on Machine Learning*, 2022.

[6] Kumar, Bordelon, Gershman, and Pehlevan. Grokking as the transition from lazy to rich training dynamics. In *International Conference on Learning Representations*, 2024. arXiv:2310.06110.

[7] Akash Kumar and Karthik Sankaranarayanan. Flat loss, evolving features: Grokking in modular addition. *arXiv preprint arXiv:2610.22593*, 2026.

[8] LeDoux. The order is the message. *arXiv preprint arXiv:2603.25047*, 2026.

[9] Ziming Liu, Kitouni, Nolte, Eric J. Michaud, Max Tegmark, and Williams. Towards understanding grokking: An effective theory of representation learning. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2205.10343.

[10] Neil Mallinar et al. Emergence in non-neural models: grokking modular arithmetic via average gradient outer product. In *International Conference on Machine Learning*, 2025. arXiv:2407.20199.

[11] Mohamadi, Li, Wu, and Danica J. Sutherland. Why do you grok? a theoretical analysis of grokking modular addition. In *International Conference on Machine Learning*, 2024. arXiv:2407.12332.

[12] Morwani, Edelman, Oncescu, Zhao, and Sham Kakade. Feature emergence via margin maximization: case studies in algebraic tasks. In *International Conference on Learning Representations*, 2024. arXiv:2311.07568.

[13] Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations*, 2023. arXiv:2301.05217.

[14] Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.

[15] Rubin, Seroussi, and Ringel. Grokking as a first order phase transition in two layer networks. In *International Conference on Learning Representations*, 2024. arXiv:2310.03789.

[16] Yuandong Tian. Provable scaling laws of feature emergence from learning dynamics of grokking. In *International Conference on Learning Representations*, 2026. arXiv:2509.21519.

[17] Tomàs, Neil Mallinar, and Mikhail Belkin. Breaking data symmetry is needed for generalization in feature learning kernels. In *International Conference on Artificial Intelligence and Statistics*, 2026. arXiv:2604.00316.

[18] Wang, Yao, Liu, Lu, and Xu. Probability-signature dynamics: Unpacking modular addition learning within two-layer networks. *arXiv preprint arXiv:2610.11833*, 2026.

[19] Yao and Xu. Probability signature: Bridging data semantics and embedding structure in language models. *arXiv preprint arXiv:2509.20124*, 2025.

[20] Ziqian Zhong, Ziming Liu, Max Tegmark, and Jacob Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.17844.

A All pre-registered tests

Table 5: Every pre-registered test and its outcome. Hashes are SHA-256 of the registration text (first 8 hex digits).

stage (hash)	test	criterion → result	verdict
v1 (`1034f794`)	C1	init+data beats each alone (CI> 0) → 1265 vs 655 / 959	**pass**
	C2	dev-selected simple statistic predicts test vote → no	**fail**
	C3	removing top datamodel pairs lowers $k^\star$, Holm $p < 0.01$ → $p = 0.0002$ (both splits)	**pass**
	C4	OV-path + data vote beats OV path → 1256 vs 576	**pass**
	C5	median commitment step ≤ 5 % of median grokking step → 2.3 % (base), 2.5 % (MLP); subtraction not testable (5/16 grokked)	**pass**
	C6	internal measures beat behavioural baseline for grokking time → C 0.85 vs 0.92	**fail**
	R1	OV path at init above chance, $p < 10^{-3}$ → 576 vs 165	**pass**
	proxy	step-500 winners contain ≥ 75 % of final keys → 56.5 %	**fail**
v2 (`9177a4a9`)	H1	vote persists with shuffled labels ($p < 0.01$) → 0.04, $p = 0.048$	**fail**
	H2	shift relabelling keeps the vote → 0.955	**pass**
	H3	vote persists for subtraction on the same pairs → 0.006	**fail**
	H4	MLP-only model keeps the vote → 0.69	**pass**
	H5	per-pair datamodel predicts the vote → held-out $R^2 \approx 0.03$	**fail**
v3 (`8292aeae`)	P1	Spearman(D(k), vote) > 0.5, $p < 10^{-3}$ → 0.709	**pass**
	P2	OV + D(k) beats OV, CI> 0 → 230 vs 154, [28, 123]	**pass**
v4 (`95c74926`)	G1	$\rho > 0.3$, $p < 0.01$ in all 4 configs → 0.65–0.74	**pass**
	G2	spread within ±25 % of $1/\sqrt{n}$ → ratios 0.89–1.01	**pass**
v5 (`8f973240`)	S1	steered early rate ≥ 0.5 and > original → 100 % vs 4 %	**pass**
	S2	dose–response Spearman > 0.5 → 0.95	**pass**
	S3	final-circuit rate steered > original, ≥ 50 % → 16/16 vs 1/16	**pass**
v6 (`2f243aaa`)	F1	$R \in [0.324, 0.602]$ → 0.289	**fail**
v7 (`318f50ae`)	T1	2-layer early steering → 100 % vs 3 %	**pass**
	T2	2-layer D(k) $\rho > 0.3$ → 0.72	**pass**
	T3	2-layer final circuit → 15/16 vs 0/16	**pass**
	T4	all-paths readout beats EOV_1 → 0.64 vs 0.68	**fail**

H1 and H3 were registered as hypotheses that the vote would *persist* when only the inputs are kept (shuffled labels) or the task is changed (subtraction). Their failure is informative: the vote is set by the equations, that is, by the group structure of input–answer pairs, not by which inputs are present.

B Seeds and settings

Dev seeds: inits and splits 0..15 (v1 dev). Test: 1000..1015 (v1 test); 5000..5015/6000..6015 (v3 splits/inits); 8000..8007/8100..8111 (v4); 9500..9507 base splits and 9600..9611 inits (v5); 7600..7607/7700..7711 (v6); 9800..9811 (v7); 2-layer pilot 9990, 9991. One-layer stage B: 10 000 steps; two-layer stage B: up to 15 000 steps, stopping 2000 steps after all runs in a batch reached 90 % test accuracy. Test accuracy was evaluated every 250 steps.